NukMiris v0.8.1 — Architecture Challenge, Round 3

Report 2 of 2 — Claude’s Revised Architecture Recommendation

Independent architect response · 28 October 2026 · Design and peer review only. The five original Claude reports are unchanged. Report 1 answers Codex’s fifteen questions (Q1–Q15); this report draws the consequences. Evidence labels are as defined in Report 1 §1.

1. Position, basis and limits

Where I now stand. Codex’s review found real defects in my proposal. Most are in my *safety claims*, not in the overall shape. I keep the shape (a transactional core, bracketed external effects, structural isolation, a rehearsed recovery) and I change the mechanisms that carried the strongest claims: retry after absence, lease-only fencing, best-effort archiving, the five-minute ordering advance, the generic-confirmation shortcut, and the RLS framing. For implementation strategy I do **not** declare a winner. My current lean and the evidence that would change it are in §8.

Limits, stated plainly.

- **Missing material:** Codex’s original architecture documents 02–05 were not provided and I cannot reach the repository from this session. My assessment of Codex’s design is limited to what 06, 07 and 08 reveal, including Codex’s own self-critique (06 §E). I make no claim about text I have not seen.
- **Not read:** the application source. Statements about existing code are Codex’s, at commit `95decc8`.
- **Verification:** I consulted public Google, Meta and PostgreSQL documentation (listed in Report 1 §1). I contacted no provider API, ran nothing, installed nothing, and changed no database or original document. A simulator pass is not used as provider evidence.

2. Valid criticisms accepted

#	Criticism (Codex 06 / 08)	My disposition	Corrected in
V1	Same-ID Calendar insert is an identity mechanism, not an established exactly-once retry contract	**Accepted.** Documentation supports a 409 for an existing ID, not a distributed guarantee. Automatic resubmission withdrawn	Report 1 Q1; SC-6
V2	A lease does not fence an already-capable stale dispatcher’s remote call	**Accepted.** Armed, nontransferable submission; no re-arming of unknown requests	Q2; SC-5
V3	The exclusion constraint allows two operations with the same appointment key	**Accepted.** One unresolved mutation per event; immutable identities; finite ranges; explicit release evidence	Q3; SC-4
V4	Five-minute uncertain-message advance changes I05	**Accepted.** Barrier restored; any relaxation is an owner decision	Q4; SC-7
V5	“Exactly one current candidate” can retarget a delayed reply to the wrong schedule	**Accepted.** Baseline history rule or exact context	Q5; SC-9
V6	Restore replay can allocate new IDs; unknown sends cannot be enumerated	**Accepted.** Deterministic identities; inert replay; gap-recomputed reminders; quarantine of all provider calls and old instances	Q6; SC-13
V7	Best-effort archive export leaves an acknowledged-withdrawal gap; the command journal is another dual write	**Accepted.** Archive-before-acknowledge; asymmetric safe defaults; T-4 split	Q7; SC-1, SC-13
V8	A STOP accepted during slow preflight is not serialized with admission	**Accepted.** Ingest-time suppression marker; re-check in the arming transaction	Q8; SC-8

#	Criticism (Codex 06 / 08)	My disposition	Corrected in
V9	`nm_biz` is not a complete ownership predicate	**Accepted.** Local identity is the authority; properties corroborate	Q9; SC-3
V10	A custom tenant setting is not authenticated authority; privileged functions and raw batches need proof	**Accepted.** RLS reframed as accident containment; hardening list; external break-glass audit	Q10; SC-11
V11	Free/busy cannot subtract the moving event; filtered sync cannot see tombstones or removed properties	**Accepted.** Event-aware check; unfiltered feed matched by local key	Q11; SC-15
V12	Three of my conversation and reminder choices deviate from baseline without owner approval	**Accepted.** Parity restored; TTL and transport gates enforced at send	Q12; SC-9
V13	A marker committed on a failed-over primary can authorize a lost send	**Accepted.** Synchronous durability for arming; epochs; quiescence; version gating	Q13; SC-14
V14	"Zero duplicate reminders" and "zero lost STOPs" were unjustified; "only stateful dependency" was inaccurate; 10× headroom was unmeasured	**Accepted.** Claims are now conditional; object archive, KMS and secrets are recognized as recovery-critical state; targets are proposals	Report 1 Q7, Q14, Q15
V15	My S-2 fallback (read-compare-write) is weaker than an ETag precondition	**Accepted.** Stop mutations rather than silently downgrade	Amendment register
V16	Duplicate ordinary replies after a restore are not "harmless"	**Accepted.** Replay-mode replies are suppressed; owner approves the trade	Q6 M4

#	Criticism (Codex 06 / 08)	My disposition	Corrected in
V17	"Multi-instance correctness is mandatory" overstated the baseline; the baseline requires current-version restart and cooperating execution	**Accepted as wording.** Horizontal deployment is a proposed topology with extra proof duties. The dispatch protocol is needed even on one host, because restarts and paused processes exist there too	SC-5

3. Where I qualify or still disagree

These are narrow. Most of Codex's review stands.

1. **Read-only uncertainty needs a resolution path.** I now agree there is no automatic retry. But a conservative default with no way to resolve it can block slots and conversations indefinitely. I add three things: continued read-only polling that adopts late events, an audited operator risk-acceptance decision, and an ETag-bump fence that converts uncertainty to proof for *conditional* Calendar mutations. Codex's review acknowledges the availability cost but, as far as 06 and 07 show, does not give a resolution procedure.
2. **Unknown scope does not have to mean the whole service.** 06 §E1 concedes that holding unknown scopes could stop a business indefinitely. I give a method to *narrow* scope: Calendar sweeps by documented reads, gap recomputation for reminders, and replay that is inert. Sends remain broadly quarantined until archive completeness is verified. I defend the narrowing but not a claim that it eliminates uncertainty.
3. **RLS is still worth keeping,** as accident containment. I accept the narrower claim; I do not accept dropping it. Its cost is low and it catches the failures that most often happen in practice (a missing predicate).
4. **Opaque server-side offers versus signed tokens.** Codex says a signed or encoded token checked against current server authority is not inherently noncompliant. Agreed. I keep server-side offers because they make I03 a lookup rather than a verification protocol, but the difference is small and either can pass.
5. **Framework choices.** Agreed that Fastify, Zod and Kysely are maintainability preferences, not safety arguments, and that a signed-ingress slice and team proficiency decide.

6. **Strict ordering has an unquantified cost.** Both of us now preserve the barrier. Meta can keep trying to deliver an accepted message for a long time, so a strict barrier could block a conversation far longer than a five-minute timeout would. I flag that cost explicitly; resolving it is an owner decision, not an engineering one.

4. Codex decisions worth adopting

Codex idea (as stated in 06/07/08)	Adopt?	Where it lands
No timer-based rearming; unresolved external requests keep protection	**Adopt**	SC-5, SC-6
Read-only reconciliation as the default for uncertain creates; no settle-window resubmission	**Adopt**	SC-6
Retain committed protections against lagging Google reads ("known-active" coverage)	**Adopt**	SC-4
Union or transfer reservation as an alternative to the exclusion constraint	**Evaluate** in E2; do not pre-select	§7.2
Preserve the uncertain-ordering barrier and old visible-message ambiguity	**Adopt**	SC-7, SC-9
Namespace deduplication identity by authenticated channel and business	**Adopt**	SC-1
Full ownership tuple at every effect	**Adopt**	SC-3
Requirement-tagged fault suite with pre-export loss, stale-worker and missing-attempt scenarios	**Adopt**	§9
Equal-scope vertical slice to compare B and C	**Adopt**	§8, E13
Anchored 0–4 "demonstrated outcome" scoring, with mandatory-gate disqualification, applied only to comparable scope	**Adopt**	§8

Codex idea (as stated in 06/07/08)	Adopt?	Where it lands
Split read-only restoration from safe write resumption	**Adopt**	T-4a/b/c
No speculative features; excluded capabilities stay excluded	**Adopt**	Scope
Honest treatment of existing implementation evidence: tests cover specified cases, not formal proof	**Adopt**	§8
Owner-decision list as a gate before implementation	**Adopt**	§10

5. Critique of Codex's architecture

Basis and limit. I have not seen Codex's documents 02–05. What follows rests on 06 §E (Codex's self-critique), 06/07's descriptions of its design, and my own inference from them. I flag inference as such and would revise this section on reading the originals.

Strengths visible from 06/07. A conservative no-rearm policy; known-active guards; explicit scope; a recovery posture that holds unknown effects; reluctance to claim proof it does not have; an equal-scope decision method; and an unusually candid self-assessment.

Weaknesses, as Codex itself reports (06 §E):

#	Weakness	Effect
1	Recovery is a conservative rule, not an executable completeness protocol; holding unknown scopes can stop a business indefinitely	Availability and operator load unquantified
2	Dispatcher termination underspecified (multiple workers in one container, half-open requests, partitions, failover losing admission evidence)	Same family as my lease problem
3	Tenant context established too trustingly	Same family as my RLS problem

#	Weakness	Effect
4	Guard representation still a sketch (exclusion, union, transfer without DDL)	Unproven under competing updates
5	Operational diagnosis lacked sentinel catalogue and commit-ordered counters	Weaker incident detection
6	Credential protection generic	Larger blast radius than needed
7	Estimates unequal and unmeasured	Not a basis for budget
8	Long-lived protections could harm availability and fairness without quantification	Potential permanent slot or conversation blocks

Additional observations (inference from 06/07; to be checked against 02–05):

1. **A resolution path for unresolved effects.** A design that holds protection until proof exists needs a defined procedure for when proof never arrives. I could not see one in 06/07. My operator risk-acceptance step and ETag-bump fence are candidates, and neither is proven.
2. **Scope discovery.** "Hold unknown scopes" is safe only if scopes can be *enumerated or bounded*. 06/07 do not show how. My Calendar sweeps and gap recomputation are one way; neither of us can enumerate sends that left with no receipt.
3. **Failover durability.** 06 §E2 lists failover losing admission evidence as underspecified. Neither design, before this round, defined synchronous durability and an epoch for armed work.
4. **Strict ordering without a duration bound.** 07 preserves the barrier pending an owner decision, which is right, but no cost estimate is given for how long it may block.
5. **A caution about the review, not the design.** 06 correctly notes that a passing test count is not proof. The same caution applies to 07's combined proposal: a design that incorporates more documented controls has not earned a higher score, which 07 itself says. I agree and apply it to my own work.

What I cannot say. Whether Codex's design is better or worse than mine. 07 reaches the same conclusion: no numerical ranking is defensible, and "equivalent" does not confer production proof on either.

6. Remaining technical disagreements and open questions

Topic	Codex (as I read 06/07)	Mine	What resolves it
Reservation arbitration	Exclusion or union/transfer; both unproven	Exclusion plus one-unresolved-mutation index plus identities	E2: same competing-update suite on both
Resolution of long-unresolved effects	Hold; escalate	Hold; poll; operator risk acceptance; ETag-bump fence for mutations	E1 (fence semantics), E9 (blocked-slot cost), owner policy
Deterministic, MAC-derived IDs versus random	Not addressed in 06/07	Derive, for discovery during replay	E11; Google guidance on ID format
Archive-before-acknowledge versus asynchronous export	Gap-detectable independent evidence, mechanism unspecified	Couple acknowledgement to archive success	E10 (latency and availability under object-store faults)
Ingest-time withdrawal marker	Serialize withdrawal with final admission	Suppress-only marker at ingest plus arming-transaction check	E5, owner decision on phrase set
RLS value	Defense in depth, not authentication	Same, narrower claim; keep it	E7
Rebuild versus selective rearchitecture	B provisional, C as equal competitor	Lean described in §8; not declared	E13
Hosting size and cost	Four-instance HA unproven	Same; depends on approved targets	Owner targets, E9

On the first row, the topic on which we most differ is not the mechanism but the *proof standard*. I would accept either design only after the same adversarial suite passes.

7. Recommended combined architecture

The shape is unchanged from my original proposal: a modular monolith over one transactional database, a pure conversation core, Google as appointment truth, structural tenant isolation, and rehearsed recovery. What changes is the **safety contract** the shape must satisfy. Nothing below is implemented or proven; each item names the evidence that would establish it.

7.1 Combined safety contract

ID	Invariant	Origin	Evidence needed
SC-1	**Accept before acknowledge.** An inbound event is acknowledged only after the database commit **and** a write to the archive have both succeeded; otherwise a non-200 is returned and the provider retries. Deduplication identity is namespaced by authenticated channel and business; a content change under one identity is a conflict, never a substitution	Claude (new) + Codex (namespacing)	E10, E6
SC-2	**Commit-ordered processing.** Per-conversation order equals commit order. Event processing, state change, queued replies and new operations commit together; replaying an unprocessed event produces the same decision with no external effect	Claude, adopted by Codex	Crash-point suite

ID	Invariant	Origin	Evidence needed
SC-3	**Authority is local.** Customer input never contains authority. Actions resolve through opaque offers to local identities. Every Calendar effect checks the full tuple (business, current Calendar mapping, customer, appointment key, unresolved operation). Event properties only corroborate	Both	E7, ownership tests
SC-4	**One unresolved mutation per event,** attendance and cancel included, enforced by a partial unique index. Reservations use finite, non-empty, half-open ranges, protect the interval from selection until evidence resolves it, and are never released by elapsed time while an armed or uncertain attempt exists. Committed protections are retained against lagging Google reads	Both	E2
SC-5	**Armed, nontransferable submission.** No provider call without a committed, durable armed attempt owned by that dispatcher. Only the owner may submit it. Lease expiry never transfers it. A new attempt follows only evidence or an audited operator decision	Codex (intent) + Claude (protocol D1–D6)	E3

ID	Invariant	Origin	Evidence needed
SC-6	**Uncertain stays uncertain.** No automatic resubmission of a Calendar create or a send. Reconciliation is read-only by deterministic identity. For conditional Calendar mutations an ETag bump may convert uncertainty into proof. Otherwise release needs positive evidence, a recorded definite refusal, or an audited operator risk acceptance	Both	E1
SC-7	**Ordering barrier.** An uncertain attempt blocks later ordinary replies in its conversation. An unsent expired prompt does not. Any relaxation is an owner decision	Codex + baseline I05	E4, E9
SC-8	**Withdrawal serialization.** An ingest-time suppression marker plus a re-check inside the arming transaction. A withdrawal wins over all unarmed work and cannot recall an armed one	Claude (new), Codex (requirement)	E5
SC-9	**Reminder rules at the last local step.** Baseline eligibility, due-through-15-minutes, start, TTL cap and transport budget enforced at arming. Exact quoted context binds to the dispatched request. Generic text acts only under the baseline history rule. Follow-up timing per baseline	Baseline, both	E5
SC-10	**Time.** Instants for offsets; civil dates in one module; DST-fold and timezone-change behaviour as the owner approves	Both	E8

ID	Invariant	Origin	Evidence needed
SC-11	**Tenant isolation.** Tenant derived from the authenticated channel; composite foreign keys; RLS as accident containment; hardened privileged functions; raw multitenant batches segregated; per-business operator grants; break-glass audit held outside the database. The blast radius of a compromised shared runtime is documented as all tenants	Both, narrowed	E7
SC-12	**Credentials.** Envelope encryption bound to business, provider and credential; short fixed cache; version and status checked in the arming transaction; separate protocols for token rotation, secret rotation and revocation	Claude + Codex	E7
SC-13	**Recovery.** PITR; evidence ledger and operator journal written before acknowledgement; deterministic identities; inert replay; reminders recomputed over the gap; quarantine covers all provider calls and old instances through credential-gated dispatch; no revival of revoked grants; split recovery objectives	Both	E6
SC-14	**Durability across failover and rollback.** Synchronous durability for arming commits; cluster epochs; post-failover reconciliation; draining; version gating	Claude (new) + Codex (E3)	E3

ID	Invariant	Origin	Evidence needed
SC-15	**Change detection.** Unfiltered change feed matched by local key, a daily direct read of every active appointment, and a preflight read before each send; event-aware reschedule check	Claude (corrected)	E8
SC-16	**Observability.** Named sentinels with a false-positive policy, commit-order and queue-age metrics, review queue, redacted correlation, canary ownership. Uncertain work is never alerted as automatically retryable	Both	Drills
SC-17	**Privacy and custody.** Data classes, retention, key and erasure policy approved by the owner and counsel; pseudonymized suppression ledger; no claim of legal compliance	Both	Owner policy

7.2 What the combined design looks like

Area	Combined design	Changed from my original?
Shape and stack	Modular monolith, TypeScript on Node, PostgreSQL; frameworks and libraries are maintainability choices, not safety arguments; no broker	No (claims about tooling softened)
Ingest	Raw-byte signature check; accept-before-acknowledge with archive; channel-namespaced dedup; per-conversation commit counter; ingest-time suppression marker; raw batches segregated	**Yes**

Area	Combined design	Changed from my original?
Conversation	Pure reducer; opaque offers; baseline command override; baseline generic-confirmation history rule; poison events fail closed	**Yes**
Calendar	Local identity authority; deterministic IDs for discovery; event-aware reschedule check; unfiltered change feed by local key; daily direct reads	**Yes**
Operations and reservations	One unresolved mutation per event; finite-range reservations; exclusion constraint evaluated against union/transfer in E2; release evidence rules	**Yes**
Dispatch	Single dispatcher per process; armed attempts; nontransferable; synchronous-commit arming; epochs; version gating; credential cache rules	**Yes (new)**
Messaging	Attempt ledger and callback correlation (subject to E4); append-only receipts and orphan review; strict barrier	**Yes**
Reminders	Baseline gates at arming; final admission transaction; read-only reconcile; gap recomputation after restore	**Yes**
Tenancy	Composite keys, RLS as containment, hardened functions, per-business operator grants, external audit	**Yes (reframed)**
Credentials	Envelope encryption with AAD; separate rotation protocols; short cache	Minor

Area	Combined design	Changed from my original?
Recovery	PITR; archive-before-ack; evidence ledger; operator journal with asymmetric defaults; inert replay; quarantine of all provider calls; credential-gated dispatch; T-4a/b/c	**Yes**
Observability	Sentinels with a false-positive policy; canary; review queue; handoff queue	Minor
Hosting	Managed containers and managed HA Postgres; object archive in a separate account; KMS and secrets recognized as recovery-critical state; size and cost pending approved targets	Softened

```
sequenceDiagram
  participant M as Meta
  participant W as web
  participant A as archive
  participant DB as PostgreSQL
  participant X as dispatcher
  participant P as provider
  M->>W: webhook
  W->>A: conditional write of raw body
  W->>DB: tx: event, sequence, withdrawal marker, evidence ledger
  W-->>M: 200 only if both writes succeeded
  X->>DB: arming tx with synchronous commit, local gates, epoch, version
  DB-->>X: acknowledged durable
  X->>P: single request, absolute deadline
  X->>DB: record outcome, or leave armed and uncertain
```

8. Rebuild versus selective rearchitecture

Options, under identical requirements and gates. A improve the current application. **B** selectively rearchitect existing components. **C** build a new implementation. For all three the scope is the same: full v0.8.1 parity, SC-1 to SC-17, the production gates from my C5 (G1–G10), and pilot-ready runbooks. NukMiris is a POC with no production customers and

no historical data to migrate, which lowers the risk of C and removes compatibility work from B. It does **not** remove the need to coordinate external side effects, provider authorization and environments.

Criterion (07's weights in brackets)	A	B	C
Reliability and correctness [25%]	Known paths are preserved and tested for covered cases. SC-5, SC-13 and SC-14 cut across storage, dispatch and recovery, so adding them in place risks two competing authorities	Replace the high-risk boundaries (storage, dispatch, delivery) while preserving verified semantics; risk is the seam during transition	Design the atomic boundaries together; every working failure behaviour must be re-established
Security and isolation [20%]	Add controls around the existing trust model	Replace trust and credential boundaries deliberately, keep ownership checks	Composite keys and roles from the start; privileged-function mistakes still possible
Maintainability and code quality [20%]	Depends on whether dispatcher, storage and presentation concerns can be separated in place	Good if seams become the final structure; poor if a permanent hybrid remains	Most coherent on paper; unproven until the team makes real changes
Testability and recovery [15%]	Existing tests are strong evidence for covered cases; destructive-loss, stale-worker and failover tests are new	Requirement-based tests survive; adapter tests change	Fixtures and scenario meanings reused; internals retested; all recovery tests new

Criterion (07's weights in brackets)	A	B	C
Scalability and extensibility [10%]	One runtime and database; no agreed load target	Staged	Cleaner start; headroom unmeasured
Operational readiness [10%]	May suit a supervised single-host pilot; recovery, support and hosting remain gaps	Can stage hosting, secrets and database topology	Can adopt the target operating model from the first slice

No numeric score. I agree with 07: assigning weighted numbers to unbuilt designs would reward documentation rather than demonstrated behaviour. After equal-scope implementations exist I would score each criterion 0–4 on demonstrated outcome with 07's anchors, and treat any mandatory safety or isolation failure as disqualifying regardless of total.

Comparable effort. Estimate each option against the same work packages, not against its own definition of done.

Work package	Where each option spends the effort
WP1 Ingest, inbox, ordering	A: refactor. B: replace. C: new
WP2 Conversation, offers, grammar	A: keep. B: keep behind ports. C: port the pure logic and fixtures
WP3 Calendar adapter, availability, identities	A: extend. B: replace the adapter, keep pure rules. C: new adapter, ported rules
WP4 Operations, reservations, dispatch protocol	A: new inside existing flow. B: replace. C: new
WP5 Messaging and receipts	A/B: extend or replace. C: new
WP6 Reminders, permission, templates	A/B: extend. C: port rules, new execution
WP7 Tenancy, credentials, operator	New in all three
WP8 Recovery, archive, restore	New in all three; deepest in A because of existing assumptions

Work package	Where each option spends the effort
WP9 Observability, sentinels, runbooks	New in all three
WP10 Provider spikes and verification suite	Identical in all three

Only a measured slice can fill these in. I cannot responsibly estimate A or B without having read the source. For C, my **judgment** is roughly **nine to fourteen engineer-months to reach the gates**, up from my earlier seven to ten because of this round's mechanisms. Codex's eight to fourteen engineer-weeks for B was, by its own account, a narrower scope. Neither number should be a budget.

My lean, with low-to-moderate confidence. The safety contract concentrates in the layers that hold persistence, dispatch, delivery and execution. Those layers are tightly coupled, so converting them in place tends to mean replacing them together. That points toward **a fresh core for those layers, with the baseline's requirement tests, fixtures and pure domain logic carried over through ports**. Whether to call that B or C is a naming question; what matters is the amount of the existing runtime that survives. Option A is least likely to satisfy SC-5, SC-13 and SC-14 without effectively rewriting the same layers, though it may suffice for a supervised single-host pilot if the owner restricts the scope. I do not declare a winner.

Evidence that would change this.

- Toward **B or A**: the slice shows existing execution paths can adopt armed attempts, identity derivation and quarantine through few seams with no dual authority; the change exercise costs no more than about 1.25 times the fresh slice; an equal-scope estimate for B is materially below C.
- Toward **C**: more than the persistence, dispatch and delivery layers must be replaced to meet the contract; seams add more risk than they remove; the team can fund parity and operations.
- Toward **A**: agreed pilot targets are modest, a supervised single-host deployment is acceptable, and the contract can be met without architectural debt obstructing routine change.
- Against all my numbers: any of S-1, S-6, S-10 or S-12 failing; measured workload outside the 10× assumption; on-call volume beyond what two people can carry.

9. Verification experiments before implementation

None has been executed. Anything touching a live Google or Meta account needs separate authorization. A simulator may precede it, but only the real provider can settle a provider question.

ID	Question	Method (design only)	Pass	If it fails
E1	Calendar identity and conditional-mutation contract (S-1, S-2)	Sandbox: concurrent same-ID inserts from two clients, lost response and delayed visibility, ID reuse after delete, late insert after cancel, ETag-conditioned patch and delete races, ETag-bump fence	Documented semantics or an explicit vendor statement, plus characterized behaviour. Samples alone never prove absence of delayed collision	Read-only default stays; stop mutations if conditional semantics are not reliable
E2	Reservation and admission	Disposable target-version database: the illustrative DDL, union/transfer alternative, two-session races, same-event moves, cancel and attendance races, NULL/empty/buffer cases, stale finalizers	Both database and operation invariants hold under competing updates	Choose the alternative that passes or add application-level admission
E3	Dispatcher and HA durability	Pause before marker, after marker before socket, after remote commit, before local finish; expire claims; fail over with a possibly lost marker; resume the old process; confirm what the managed service guarantees	No second possible submission; no released uncertainty; failover loses no acknowledged arming	Treat failover as disaster recovery; choose a different service tier

ID	Question	Method (design only)	Pass	If it fails
E4	Meta evidence and order (S-6, S-7)	Simulator first, then sandbox: callback echo on **free-form and template** messages (the changelog documents free-form), missing echo, duplicate and reordered receipts, two sends to one recipient, error-code classes	Unknown correlation stays unresolved; no silent order change	More sends stay uncertain; strengthen operator tooling
E5	Reminder races	STOP during slow reads; cancel or move after claim; template status or TTL change; delayed parent receipt; stale unquoted confirmation after a move	Exact consent, target, mode and TTL boundaries held	Tighten arming gate
E6	Destructive-loss restore	Snapshot; accept a STOP and an operator revoke; dispatch a no-receipt send and a Calendar change; destroy the database before archive export, with and without archive-before-ack; replay with deterministic identities; fence an old instance	No invented IDs; no reactivated permission; unknown scopes quarantined; read-only restoration separate from safe write resumption	Keep writes blocked; revisit archive design

ID	Question	Method (design only)	Pass	If it fails
E7	Tenant privilege	Real roles: arbitrary `SET`, `PUBLIC` function execution, cross-business batches, FK substitution, AAD swap, pooled-connection reuse, break-glass audit held externally	Bug-defence demonstrated; compromised-runtime exposure documented	Add per-tenant credentials or narrow claims further
E8	Search, sync, phone, time (S-3, S-4, S-9, S-11)	Merged busy events during a move; all-day, transparent and declined events; property removed; tombstones; pagination and token rewind; Mexico and Argentina identity versus display; tzdata upgrade	No false free time; no retargeted identity; no missed deletion	Block on ambiguity; widen direct reads
E9	Capacity, operations, economics (S-5, S-8, S-12)	Approved workload model; bursts; hot tenants; quotas; archive lag; failover and restore time; cost of blocked slots and conversations; complete monthly cost	Observed headroom and costs within agreed targets	Revise targets or topology
E10	Archive-before-acknowledge	Latency and availability under object-store faults; reliance on provider retry; duplicate writes; manifest gap detection; database-loss drill before export	Acknowledged events always recoverable; acceptable latency	Asynchronous export with explicit suppression on gaps

ID	Question	Method (design only)	Pass	If it fails
E11	Deterministic identity	Provider acceptance of derived IDs; format and collision guidance; inert replay with derived IDs	Replay reads, never creates	Random IDs plus an identity ledger kept out of band
E12	Withdrawal fast path	Compare the phrase set and protected-entry behaviour with the baseline (code and test reading, no execution)	No false suppression beyond an agreed bound	Drop the fast path; serialize in the reducer under a lock
E13	Equal-scope slice: B versus C	Same vertical slice under both; metrics from Report 1 Q15	Decision rule in §8	Choose by rule; record scores by 07's anchors

10. Questions for the product owner

Each has my recommended default. A default is a parity-first suggestion, not a decision.

1. **Calendar capacity and sharing.** Is capacity exactly one appointment per Calendar, and may two businesses share a Calendar? *Default: one and one.* What is the policy when a manual or third-party event overlaps a NukMiris booking (U-EXTERNAL)?
2. **WhatsApp account ownership.** Who owns the WABA and its grants, who pays, who maintains templates? Not a request to build billing.
3. **Ordering barrier.** Pure barrier with operator release, a long owner-approved maximum hold, or self-contained version-stamped replies? *Default: pure barrier.*
4. **Unresolved Calendar creates.** May an operator accept residual risk and release a held slot, with what authority, SLA and maximum blocked duration? *Default: yes, audited, two-person if staffing permits.*
5. **Conversation edge policies.** Command and name collisions; expired name prompts; poison events; historical-outcome wording; DST fold and future timezone-rule changes. *Default: baseline parity; any change approved explicitly.*
6. **Generic confirmation.** Keep the baseline history rule, or approve a bounded alternative only after a demonstrated safe rule? *Default: baseline.*

7. **Late parent receipt.** Skip at first processing (baseline) or wait within the allowance? *Default: baseline.*
8. **Withdrawal.** Exact phrase set, interaction with protected name entry, and acceptance of a suppress-only marker at ingest.
9. **After a restore.** Is silence preferable to a repeated ordinary reply? *Default: silence.*
10. **Privacy and custody.** Retention classes, pseudonymized evidence, crypto-shredding, residency, the archive in a separate account, and how external Calendar copies of personal data are handled.
11. **Targets and budget.** Workload and bursts; T-4a, T-4b and T-4c; ingress versus transaction-success objectives; support hours; who is on call; the monthly cost ceiling.
12. **Operator model.** Who may act on which business, offboarding, second approver for risky actions.
13. **Pilot scope.** Is a supervised pilot on a narrower scope acceptable, and if so which gates apply?
14. **Authorizations.** Provider spikes (E1, E4) and the B-versus-C slice (E13): budget and permission.

11. What neither architecture has solved

1. **Provider contracts.** Google gives no documented distributed exactly-once guarantee for client-assigned IDs, and Meta gives no documented send-side idempotency or ordering guarantee that I could find. Everything built on those is mitigation.
2. **The frozen-process window** between the final conditional update and the socket write. Bounded to one request on a protected interval, not eliminated.
3. **The cost of indefinite uncertainty:** blocked slots, blocked conversations, operator workload. It is a policy and staffing question as much as an engineering one.
4. **Manual and third-party writers.** Detected and flagged, never prevented.
5. **Human handoff.** The system records it. Nobody is staffed to answer it.
6. **Restore completeness** when the database and archive are lost together, and for sends that left with no receipt.
7. **Personal data held in the business's own Calendar,** which NukMiris does not control, and the legal minima for consent evidence.
8. **Customer experience** of late or stale messages, which safety validation makes harmless but not pleasant.
9. **Measured capacity, cost and staffing.** Every number in both proposals is a judgment.

10. **Time-zone rule changes** for appointments already written to Google.

12. Closing

I have answered all fifteen questions in Report 1 and carried the consequences here. I have not edited any original report, written code, changed a database, installed anything, or contacted a live Google or Meta API. I have not declared a winner. The architecture I would support is the combined contract in §7, accepted only after the experiments in §9 pass and the owner decisions in §10 are made. I could not review Codex’s documents 02–05 and would revise §5 if given them.

Complete. Awaiting further instructions.